=== ЗАПУСК КОРРЕКТНОГО ТЕСТА ENCODER-DECODER ===
максимальные размеры локальной группы для устройства на основе варпа:X=256,Y=4
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-76b251.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-e6cd6f.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-50d45d.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-42fea7.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-420ccf.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-354208.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-4aa640.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-f2b38c.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-c5a4cd.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-b5cdfa.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-13ea82.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-b3cbf2.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-8d466d.d
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-b59d60.d

[Успех] Все тензоры параметров внесены в Adam.
---
Could not open file: C:\Users\╨Ф╨╛╨╗╤B╤A╨╡╨╝\AppData\Local\Temp\dep-c93ba1.d
Батч 01/7 | MSE Loss: 8,789532 | Скорость: 763 токенов/сек(0,06360603379303344 it/s)
Батч 02/7 | MSE Loss: 8,782935 | Скорость: 691 токенов/сек(0,05761351876789757 it/s)
Батч 03/7 | MSE Loss: 8,728963 | Скорость: 624 токенов/сек(0,05199987921468056 it/s)
Батч 04/7 | MSE Loss: 8,702959 | Скорость: 695 токенов/сек(0,05792955607254595 it/s)
Батч 05/7 | MSE Loss: 8,676693 | Скорость: 731 токенов/сек(0,06091994712611581 it/s)
Батч 06/7 | MSE Loss: 8,622449 | Скорость: 709 токенов/сек(0,059109024260441935 it/s)
Батч 07/7 | MSE Loss: 8,550838 | Скорость: 741 токенов/сек(0,06176024581170732 it/s)
---
средняя скорость обработки:0,05989201538501899 it/s
---
Эксперимент успешно выполнен!

Отлично , вынеся редукцию мы можем приступить к еще 1 оптимизация lstm проходов перевернув их в некотром смысле  »с ног на голову!»

```
activeQueue.EnqueueNDRangeKernel(
    _kernelLstmForward,
    new IntPtr[] { (IntPtr)batchSize, (IntPtr)wgSizeY },
    new nint[] { (IntPtr)1, (IntPtr)wgSizeY }
);
```

Сейчас у нас исполдьзкеть не больше 8*1024 потока ! (то ничтожно мало по меркам gpu , а учитовая что hiddensize может быть больше 1000 создает немалые тормоза.
Моя идея сводиться к следующим концепциям:
1. Мы задействуем 3 измерения и id батча будет браться по 3 измерению ( поскльку Z группа самая маленькая , а батчей врдяли будет больше 16)!
2.Мы применим хитрый трюк двух осей к следующим циклам:

```
for (int t = 0; t < seq_len; t++) {
    int x_offset = b * (seq_len * in_features) + t * in_features;
    int ws_offset = t * (batch_size * ws_stride) + b * ws_stride;
    int h_out_offset = b * (seq_len * hidden_size) + t * hidden_size;

    // 1. Матричные умножения для всех 4 гейтов (каждый поток берет свои hid)
    for (int hid = lid; hid < hidden_size; hid += wg_size) {
      for (int g = 0; g < 4; g++) {
        int gate_idx = g * hidden_size + hid;
        float val = bias[gate_idx];

        int i = 0;
        for (; i <= in_features - 4; i += 4) {
          float4 x_vec = vload4(0, X + x_offset + i);
          float4 w_vec = (float4)(W[(i + 0) * (4 * hidden_size) + gate_idx],
                                  W[(i + 1) * (4 * hidden_size) + gate_idx],
                                  W[(i + 2) * (4 * hidden_size) + gate_idx],
                                  W[(i + 3) * (4 * hidden_size) + gate_idx]);
          val += dot(x_vec, w_vec);
        }
        for (; i < in_features; i++) {
          val += X[x_offset + i] * W[i * (4 * hidden_size) + gate_idx];
        }

        int h = 0;
        for (; h <= hidden_size - 4; h += 4) {
          float4 h_vec = vload4(0, h_curr_ptr + h);
          float4 u_vec = (float4)(U[(h + 0) * (4 * hidden_size) + gate_idx],
                                  U[(h + 1) * (4 * hidden_size) + gate_idx],
                                  U[(h + 2) * (4 * hidden_size) + gate_idx],
                                  U[(h + 3) * (4 * hidden_size) + gate_idx]);
          val += dot(h_vec, u_vec);
        }
        for (; h < hidden_size; h++) {
          val += h_curr_ptr[h] * U[h * (4 * hidden_size) + gate_idx];
        }

        workspace[ws_offset + gate_idx] = val;
      }
    }
```

Если на хосте выделить временный val массив размернсти get_global_size(0)^2 *batch_size(котрое >batch_size но меньше или hiddeen_size)
то можно распараллелить цикл сразу по 2 осям:
если get_global(1)==0 то мы считаем ыумму произвдения X*W и затем раскладываем по val[get_global_id(2)*get_global_id(0)+k].
Затем идет барьер , а дальше каждый get_global_id(1) прибавляет к своей val порции данных (поселдние циклы по hidden_size).
Затем идет баоьер и переорс данных в workspace ( с обратным проходом аналогично).